Where 4-Bit Quantization Breaks Small Tool-Calling Agents, and Which Repairs Fix It

CMPE 295A · Advisor review · November 13, 2026

Ekant Kapgate

Vritika Malhotra

Yashashav Devalapalli Kamalraj

Advisor: Dr. Bertin Cordova-Diba

M2 status · references, parity and cluster capacity · decisions for you · plan to M3 (Dec 4)

Where we are against M2, M3 and Dec 4

M2 (references + plan)

BF16 references: Qwen3 0.6B / 1.7B / 4B done (2,502 / 2,502 / 2,504 of 2,505; the rest excluded by rule)

New models, BF16 (finished, FR-E2): LFM2.5-1.2B 2,495 (99.6%), 2,494 / 2,494; LFM2.5-2.6B 2,504 (99.96%), 2,504 / 2,504; SmolLM3-3B 2,458 + 47 excluded (98.1%, question 10), 2,455 / 2,456 (question 11)

FR-E2: 7,506 / 7,506 verdicts agree (Qwen3)

S10 rerun on the merged code (array 111842): 104 of 104 episodes

Statistics: planted-effect check passes; real data (BF16 vs Q4_K_M, 2,000 pairs): models A–C fit, A's rate ratio 0.95 [0.91, 0.98]

Pre-registration: review fixes in; commit hashes filled (provisional); decisions marked proposed until you confirm

Needs the team: merge to main, human labels (blank kit ready), OSF filing

M3 (damage map v1, Dec 4)

KL vs BF16 on CUDA and CPU: 6 models x 6 formats (the KL exit item is met)

H1 damage runs, Qwen3-0.6B (arrays 111846 / 111848 / 111850, packs 111931 / 112078 / 112163): Q4_0 2,501 + 4 excluded, Q4_K_M 2,504 + 1, IQ4_XS 2,498 + 7 of 2,505 episodes

Qwen3-1.7B: Q4_0 2,274 + 91, Q4_K_M 2,496 + 9, IQ4_XS 1,630 + 4; LFM2.5-1.2B: 0 + 0, 0 + 0, 0 + 0

R1 grammars: JSON and Pythonic, property-tested with llama.cpp's validator

R2a variants built. R2b for Qwen3-0.6B built: 78 of 196 tensors at Q8_0, 5.56 bits/weight (FR-E10 passes); WikiText KL 0.102 vs 0.061 for uniform Q5_0 at the same size. 1.7B and LFM2.5-1.2B: base logits done, rankings queued (jobs 112005–112014)

Label kit: 30 + 300 blank sheets, rater guide, kappa script

Dec 4 (Canvas)

Report, chapters 1-2: outline in the Canvas template; draft to you Tue Nov 24

Prototype demo: storyboard ready; format is question 6

Project contribution: CONTRIBUTORS.md records who ran what this week

Meeting log 2: Oct 30 and this week's notes by the attendees

Episode counts: scripts/run_status.py -> results/m3/run_status.json (development data). Week status: progress/drafts/2026-11-09_week_plan_Nov9-13.md section 0.

Our agent: one tool-calling model in a reason → act → observe loop

Reason
the model reads the conversation and the tool schemas

↓

Act
it calls one or more tools, or answers in text

↓

Observe
each tool result is appended; next step

Category: a single tool-calling (function-calling) agent

ReAct-style loop (Workbook 2 Ch. 1). The small model under test is the only agent; it decides every action.

Purpose: an on-device assistant that acts through APIs

Files, car controls, travel, trading, messaging; airline, retail and telecom customer service. 0.6B–4B models that fit a laptop or phone at 4 bits.

One step = one generation

Chat template with tools → llama-server → parse calls → execute → log; ≤ 20 steps per turn, ctx 40,960, ≤ 4,096 new tokens.

docs/agents_and_data.md §1; testbed/runner.py (C2), testbed/backends.py (C3), testbed/prompt_formats.py.

One agent; the simulated user, tools and judges are the environment

Agent (the model under test)

Qwen3, SmolLM3 or LFM2.5 as a GGUF file on llama-server, CUDA or CPU. Its prompt format is its own chat template.

Tool executors

BFCL: the benchmark's Python API classes. τ^2: the domain database environment. Deterministic.

User

BFCL: scripted turns from the task. τ^2: an LLM simulated user (gpt-oss-20b, temperature 0, one cluster job), the same for every model and format.

Judge

Official checkers, unmodified: bfcl-eval's multi-turn checker and τ^2's database-state evaluator.

Not a multi-agent system

Only the agent changes between configurations. The simulated user is a second LLM, but it is part of the benchmark, fixed and cached (C4), so a replay of an episode needs no simulator call.

docs/agents_and_data.md §1 (C1, C4, C6 components; FR-E2 agreement in results/m2/evaluator_agreement.json).

Architecture: what each layer is for, with our numbers

Workload

C1 benchmark adapter · C4 simulated user

Same 501 tasks for every cell

Execution

C2 runner · C3 backend · C7 repairs · C8 QAD

Real llama.cpp 4-bit kernels

Measurement

C5 step logger · C6 classifier · C9 cost

Every step logged and labelled

Analysis + control

C10 stats · C11 config, registry, Slurm

Resumable; run id = hash

Hardware (CoE HPC, Slurm)

A100 40 GB: 1,578 episodes/h (4 tasks, at most 2 at once); 2.4 s per 0.6B BFCL episode

P100 12 GB: 182 episodes/h on one GPU; 15.5 s per episode

H100 80 GB: QAD training (R3)

CPU nodes: the CPU backend arm, KL on CPU, R

QoS: 2 GPU nodes per user, so our pace is set by nodes, not GPUs

CUDA vs CPU, same GGUF

Qwen3-0.6B: 90.5% identical at BF16, 53.0% at Q4_0

Qwen3-1.7B: 92.0% identical at BF16, 66.5% at Q4_0

Qwen3-4B: 96.0% identical at BF16, 67.0% at Q4_0

Below 99%: backend stays a factor

Memory per format

Qwen3-4B: BF16 7.50 GiB, Q4_K_M 2.33 GiB, Q4_0 2.11 GiB

Bits per weight: Q4_0 4.51, IQ4_XS 4.52, Q4_K_M 4.97, Q8_0 8.51

docs/architecture.md; results/m2/throughput.json, results/m2/parity_qwen3-*.json, results/m3/models_manifest.json.

Data: 8 BFCL API domains and 3 τ^2 customer-service domains

BFCL API class (domain)	Functions	Tasks (base / miss_func / miss_param)
VehicleControlAPI	22	26 / 26 / 27
GorillaFileSystem	18	20 / 28 / 27
TravelAPI	18	25 / 28 / 21
TradingBot	20	29 / 18 / 25
TwitterAPI	14	19 / 21 / 19
MessageAPI	10	16 / 21 / 20
TicketAPI	9	16 / 16 / 14
MathAPI	17	12 / 14 / 15

τ^2 domain	Base	Write tasks run	Agent / user tools
airline	50	26	14 / 0
retail	114	104	16 / 0
telecom	114	71	13 / 30

501 tasks × (4 sampled + 1 greedy) = 2,505 episodes per cell

BFCL: a fixed-seed 100 of 200 tasks per category. miss_func withholds a function; miss_param leaves out a value (asking is right).

τ^2: only tasks whose gold actions write to the database (an agent that does nothing would pass the rest).

Other data: WikiText-2 test (KL), WikiText-2 train minus 13-gram overlaps (imatrix), a 50.0M-token QAD mix (5 Hugging Face sources), all filtered against the benchmark text.

docs/agents_and_data.md §2–4 (counts from bfcl-eval 2026.3.23 and tau2-bench-verified 864350a data).

What measures the agent: verdicts and step classes

Failure class (codebook v0)	Qwen3-0.6B BF16, per 100 steps
tool_error_payload	9.4 ± 0.5
exec_error	7.1 ± 0.6
unknown_tool	6.9 ± 1.3
missing_call_direct_answer	2.9 ± 0.4
invalid_args	2.7 ± 0.5
missing_call	1.0 ± 0.2
malformed	0.3 ± 0.1
forced_quit	0.0 ± 0.0

Episode verdict: the official checkers

bfcl-eval multi-turn checker (state + response) and τ^2's database evaluator.

FR-E2: our re-scoring agrees on 7,506 of 7,506 Qwen3 reference episodes.

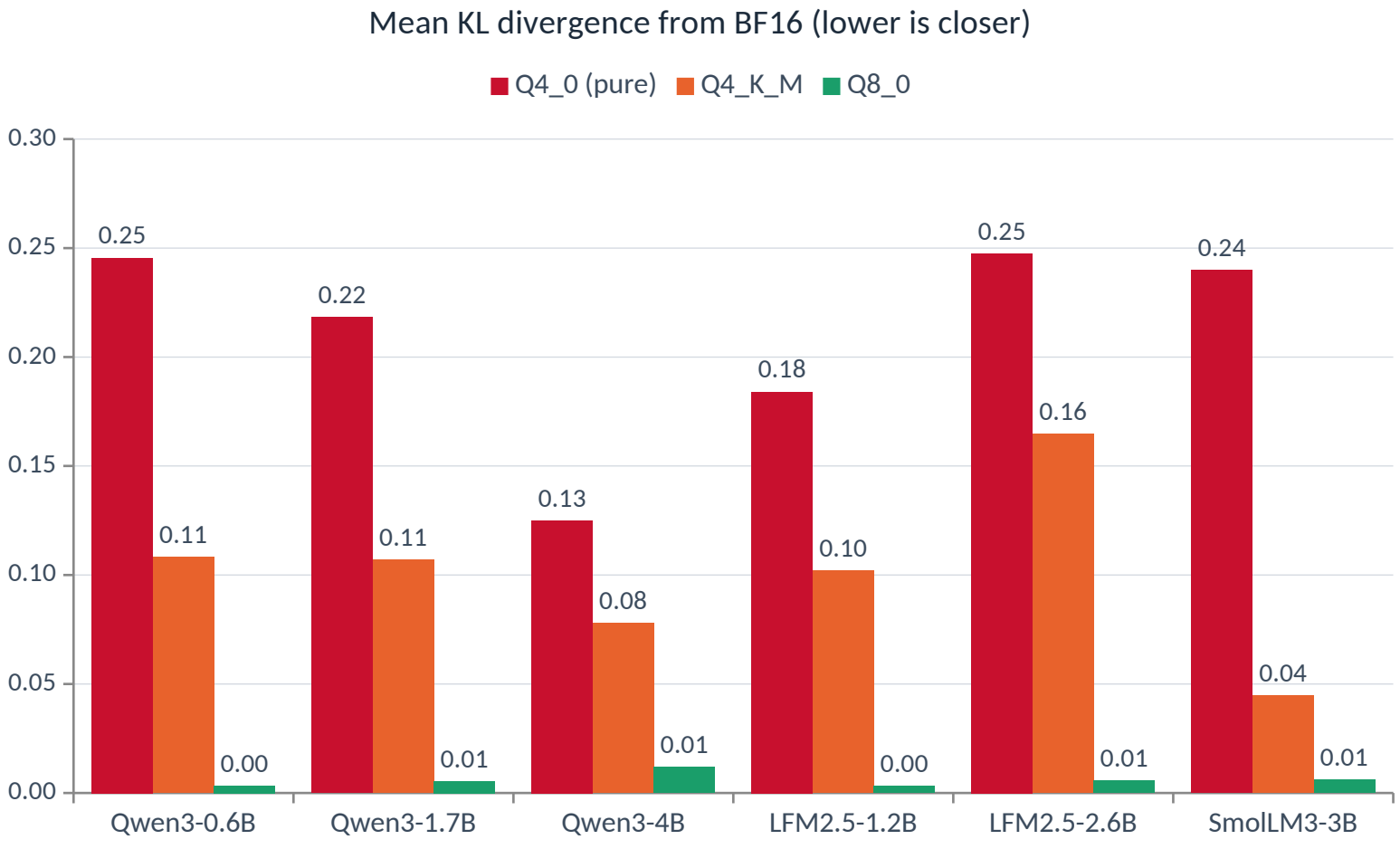
Step class: what went wrong, and where

13 classes, first matching rule wins (format before content). Primary outcome: failures per step (decision 2).

Not covered in v0: wrong tool, wrong value, policy violation (need the reference trajectory); raters note them for v1.

docs/codebook_v0.md; results/m3/which_tasks_fail_qwen3-0.6b.json (sampled seeds, mean ± SD over seeds; development data); results/m2/evaluator_agreement.json.

Six models, four formats: Q8_0 stays close to BF16, 4-bit does not



Same top token as BF16

Q8_0: 94.8–96.2%

Q4_K_M: 81.1–88.9%

Q4_0: 74.7–85.7%

Cite KLD, not perplexity

LFM2.5-2.6B Q4_K_M lowers WikiText perplexity (ratio 0.86, rechecked with 200 chunks) while its KLD is 0.165.

llama-perplexity --kl-divergence vs each BF16 file; WikiText-2 raw test, ctx 512, 50 chunks, CUDA on a P100. Q4_K_M ≈ 5.0 bits per weight, Q4_0 ≈ 4.5. results/m2/kl_*.json, models_manifest.json.

BF16 references: even 16-bit agents rarely finish a task

Model	BFCL base	miss_func	miss_param	τ^2 airline	τ^2 retail	τ^2 telecom
Qwen3-0.6B	4.5% (6.0)	2.0%	1.2%	0%	1.9%	0%
Qwen3-1.7B	6.2%	2.2%	8.5%	0%	3.6% (1.9)	0%
Qwen3-4B	12.2%	9.5%	11.0%	3.8% (0)	8.2% (10.6)	1.1%

Episode success over 4 sampled trials; greedy in brackets where it differs by more than 1 point. 501 tasks per model: BFCL multi-turn 3 × 100 + τ^2 write tasks (airline 26, retail 104, telecom 71). 2,505 episodes each.

So the primary outcome is per step (decision 2)
With success near zero, an episode-level comparison of 4-bit vs BF16 has almost nothing to compare. Failures per step exist in every episode, solved or not; episode success stays as the secondary outcome.

7,508 / 7,515

episodes finished; 7 excluded by the empty-reply rule

4 + 1

sampled trials (T 0.7, top-p 0.8, top-k 20) + 1 greedy pass

40,960

token context, 4,096 new tokens per step (bfcl parity)

Runs ref_qwen3_0p6b/cd314594fb, ref_qwen3_1p7b/c456da2aef, ref_qwen3_4b/1c7b26d4e7 (2,505 specs each). Development data. docs/cluster.md, M2 results 2026-11-09.

Our scoring matches the official evaluators on every Qwen3 episode

7,506 / 7,506

verdicts identical to bfcl-eval and τ^2-bench when every finished Qwen3 reference episode is re-scored from its logged trajectory (FR-E2: PASS)

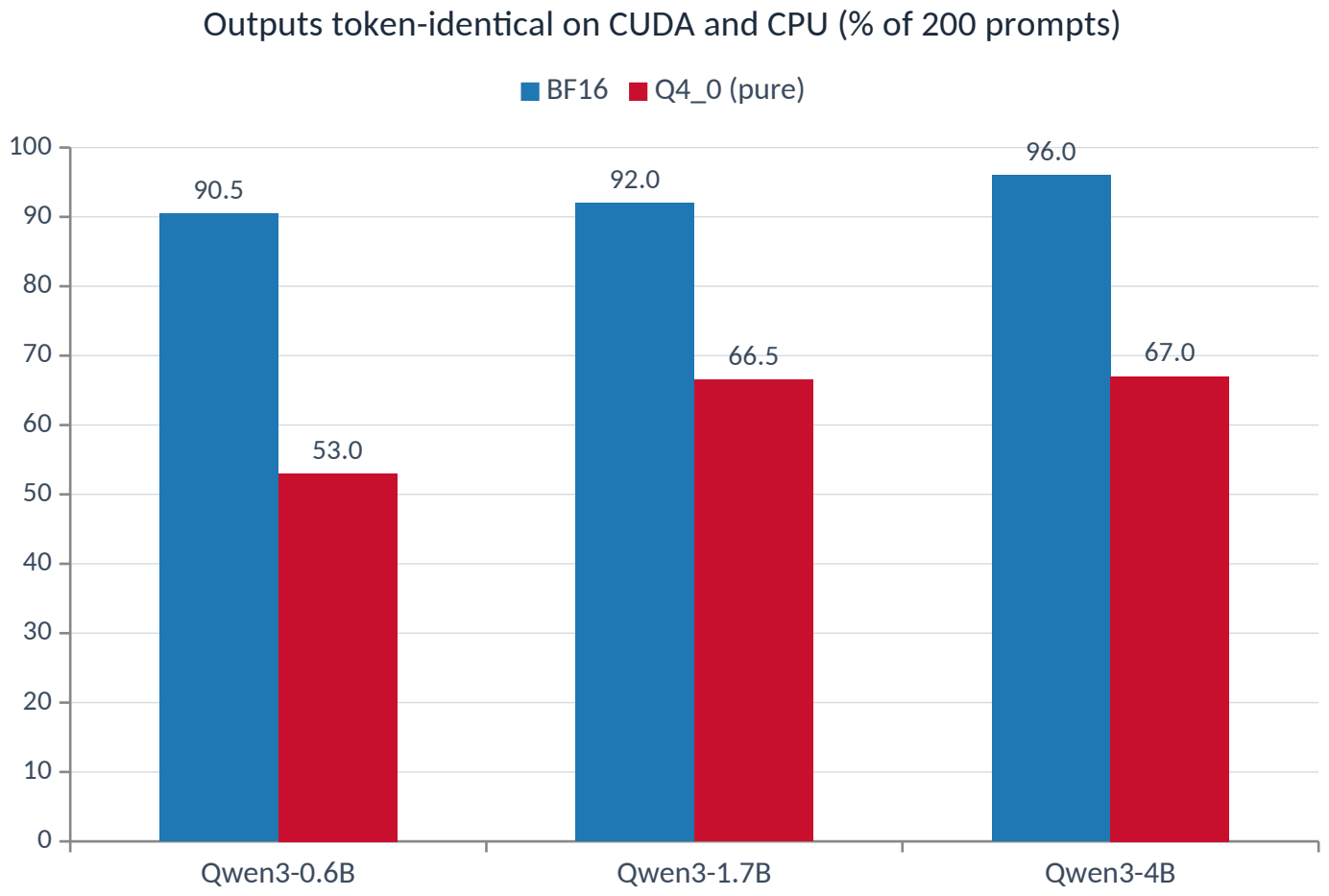
BFCL: 4,500 of 4,500
Re-scored through bfcl-eval's own path for a local Qwen3 FC model: its response parsing, decode_execute and multi-turn checker.

τ^2-bench: 3,006 of 3,006
Re-scored with τ^2's evaluate_simulation on the saved simulations.

Two episodes not re-scorable
Two τ^2 episodes ended in a context overflow (one agent, one simulated user) and have no simulation to re-score; they are scored as failures without the evaluator.

scripts/evaluator_agreement.py, CPU job 110190; results/m2/evaluator_agreement.json. bfcl-eval's API-client imports are replaced with inert stand-ins (listed in the JSON); multi-turn scoring never calls them.

CUDA and CPU agree less at 4 bits, so the backend stays a factor



100%

of prompts tokenize identically on both backends

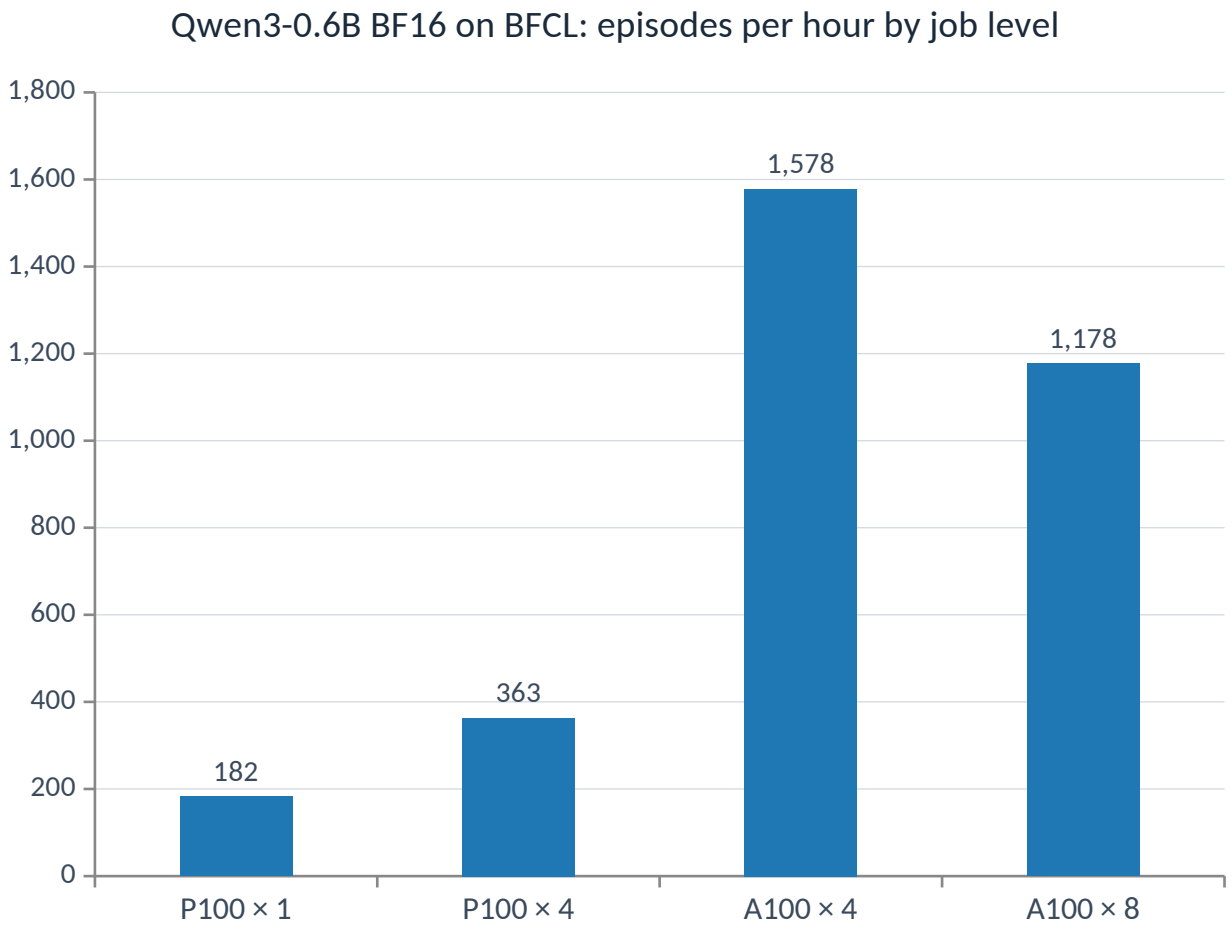
97–100%

same call-or-reply decision (Q4_0 97.0–98.5%)

Pooling rule not met (decision 7)
We pool backends only at ≥ 99% identical outputs. No model reaches it, so backend is a recorded factor in every RQ1 analysis.

FR-E17: 200 fixed prompts per model (Oct 28 BF16 episodes replayed), greedy, ctx 40,960, cache off, same GGUF file. CUDA on P100 (4B BF16 on H100), CPU on cpuqs, 32 CPUs. results/m2/parity_qwen3-*.json.

The 2-node limit per user, not the GPUs, sets our pace



Level	Jobs ran at once	Median episode	Queue wait
P100 × 1	1	15.5 s	54 min
P100 × 4	2	16.5 s	65 min
A100 × 4	2	2.4 s	167 min
A100 × 8	2	2.3 s	175 min

NFR-5 (≥ 8 concurrent jobs) is not met
QoS gpu3nodes allows 2 nodes per user and the A100 nodes are shared; episodes per hour (≥ 120) passes easily. Plan: 3–7 concurrent A100 jobs. Ask: can you request a higher node limit, or may shares run from our other accounts?

scripts/throughput_submit.sh, 24 episodes per job; arrays 109478–109481. results/m2/throughput.json. A100 nodes cs001 and cs002 excluded (bad GPUs).

What the main grid can detect at 2,000 episodes per configuration

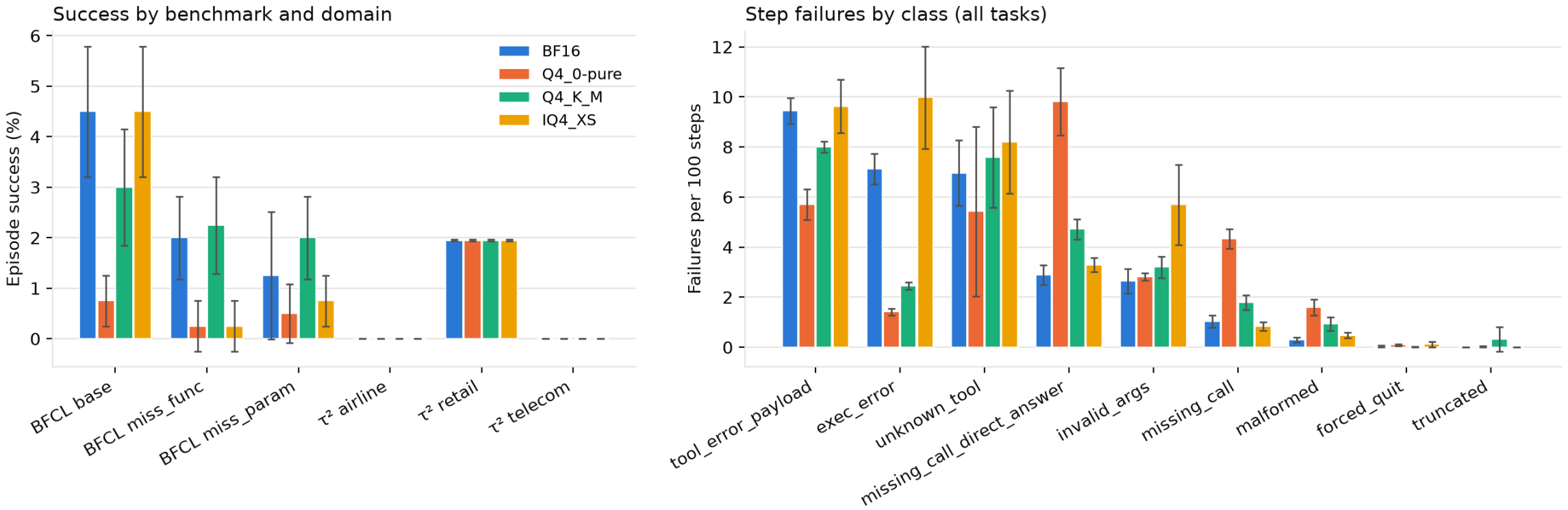
Failure class	Rate per step (0.6B / 1.7B / 4B)	Detectable?
any failure	30% / 27% / 23%	yes, all models
exec_error	7.2% / 10.4% / 6.6%	yes
tool_error_payload (new)	9.4% / 8.0% / 6.2%	yes
unknown_tool	7.0% / 5.5% / 8.5%	only at low clustering
missing_call	1.0% / 1.4% / 0.9%	borderline
missing_call_direct_answer (new)	2.9% / 0.5% / 0.6%	0.6B only
invalid_args	2.7% / 0.9% / 0.2%	no
malformed, forced_quit, truncated	< 0.3% each	no: pool or explore

Tasks the BF16 reference solves
0.6B: 17 · 1.7B: 33 · 4B: 73 of 501
All under the registered 120, so success-based inclusion applies only to the secondary analysis (decision 11).

Proposal for the pre-registration
Confirmatory: any failure, exec_error, tool_error_payload. Exploratory or pooled: the rarer classes.

Registered test: 4-bit share of events 0.5 vs 0.6, α = 0.05/10, 80% power → 330 events at design effect 1. Verdict uses design effects between task and episode clustering. results/m2/power_table.json, inclusion_qwen3-*.json.

Which tasks fail: BF16 vs 4-bit (development data)



qwen3-0.6b: sampled trials (seeds 1-4), bars = mean over seeds, whiskers = ± 1 SD. All bars on the same 1993 (task, seed) pairs. Development data, preview.

analysis/which_tasks_fail.py; results/m3/which_tasks_fail_qwen3-0.6b.json; 1,993 common (task, seed) pairs, configurations BF16, Q4_0-pure, Q4_K_M, IQ4_XS. Development data, preview.

First 4-bit damage numbers, Qwen3-0.6B (preview)

Failure class	BF16 per 100 steps	Q4_0-pure: per 100 steps · ratio [95% CI]	Q4_K_M: per 100 steps · ratio [95% CI]	IQ4_XS: per 100 steps · ratio [95% CI]
any failure	30.5	31.1 · ×1.02 [0.95, 1.10]	29.0 · ×0.95 [0.90, 1.00]	38.2 · ×1.25 [1.19, 1.31]
exec_error	7.2	1.4 · ×0.20 [0.14, 0.26]	2.4 · ×0.34 [0.28, 0.41]	9.9 · ×1.39 [1.24, 1.57]
missing_call_direct_answer	2.9	9.8 · ×3.40 [2.91, 4.06]	4.7 · ×1.64 [1.43, 1.89]	3.3 · ×1.14 [1.00, 1.30]
tool_error_payload	9.4	5.7 · ×0.60 [0.53, 0.68]	8.0 · ×0.85 [0.78, 0.92]	9.6 · ×1.02 [0.92, 1.13]
unknown_tool	7.0	5.4 · ×0.78 [0.62, 0.95]	7.6 · ×1.08 [0.98, 1.21]	8.2 · ×1.18 [1.03, 1.35]
invalid_args	2.7	2.8 · ×1.05 [0.78, 1.42]	3.2 · ×1.19 [0.94, 1.54]	5.7 · ×2.14 [1.60, 2.84]
missing_call	1.0	4.3 · ×4.22 [3.37, 5.44]	1.8 · ×1.75 [1.43, 2.17]	0.8 · ×0.81 [0.65, 1.02]
malformed	0.3	1.6 · ×5.40 [3.44, 9.56]	0.9 · ×3.19 [1.84, 5.93]	0.5 · ×1.60 [0.97, 2.81]
truncated	0.0	0.0 · –	0.3 · –	–

95% CI above 1: malformed ×5.4 (Q4_0-pure); missing_call ×4.2 (Q4_0-pure); missing_call_direct_answer ×3.4 (Q4_0-pure); malformed ×3.2 (Q4_K_M); missing_call ×1.7 (Q4_K_M); missing_call_direct_answer ×1.6 (Q4_K_M); any_failure ×1.3 (IQ4_XS); exec_error ×1.4 (IQ4_XS); forced_quit ×3.0 (IQ4_XS); invalid_args ×2.1 (IQ4_XS); missing_call_direct_answer ×1.1 (IQ4_XS); unknown_tool ×1.2 (IQ4_XS)

95% CI below 1: exec_error ×0.20 (Q4_0-pure); tool_error_payload ×0.60 (Q4_0-pure); unknown_tool ×0.78 (Q4_0-pure); exec_error ×0.34 (Q4_K_M); tool_error_payload ×0.85 (Q4_K_M)

Development data from the three complete cells; not the registered test. Fewer exec_error and tool_error_payload steps under 4-bit can mean fewer calls made at all.

analysis/damage_table.py; results/m3/damage_preview_qwen3-0.6b.json. Ratio = 4-bit rate / BF16 rate per step on paired episodes (Q4_0-pure: 1,999 pairs, Q4_K_M: 2,000 pairs, IQ4_XS: 1,995 pairs); task-cluster bootstrap, 10,000 resamples. Not shown (lowest rates): forced_quit; see the JSON.

Example from our logs: same BFCL task and seed, BF16 vs Q4_0

BF16: verdict pass

t0.s0 ok_call: <tool_call> {"name": "place_order", "arguments": {"order_type": "Buy", "symbol": "AAPL", "price": 150.0, "amount": 100}} </tool_call>

→ {"order_id": 12446, "order_type": "Buy", "status": "Pending", "price": 150.0, "amount": 100}

t0.s1 end_turn: The order for 100 shares of AAPL at $150 each has been placed successfully with an order ID of 12446. The status is "Pending". Let me know if you need [...]

t1.s0 ok_call: <tool_call> {"name": "get_order_details", "arguments": {"order_id": 12446}} </tool_call>

→ {"id": 12446, "order_type": "Buy", "symbol": "AAPL", "price": 150.0, "amount": 100, "status": "Open"}

t1.s1 end_turn: The detailed information for your order with ID 12446 is as follows: - **Order Type**: Buy - **Symbol**: AAPL - **Price**: $150.00 - **Amount**: 100 [...]

t2.s0 ok_call: <tool_call> {"name": "cancel_order", "arguments": {"order_id": 12446}} </tool_call>

→ {"order_id": 12446, "status": "Cancelled"}

t2.s1 end_turn: Your order for 100 shares of AAPL at $150.00 has been successfully cancelled. The status is now "Cancelled". Let me know if you need further assistanc [...]

Q4_0-pure: verdict fail (multi_turn:inference_error)

t0.s0 ok_call: <tool_call> {"name": "place_order", "arguments": {"order_type": "Buy", "symbol": "AAPL", "price": 150, "amount": 100}} </tool_call>

→ {"order_id": 12446, "order_type": "Buy", "status": "Pending", "price": 150.0, "amount": 100}

t0.s1 end_turn: The order has been placed successfully. The order ID is 12446, status is Pending, price is $150.0, and 100 shares have been executed at this price.

t1.s0 ok_call: <tool_call> {"name": "get_order_details", "arguments": {"order_id": 12446}} </tool_call>

→ {"id": 12446, "order_type": "Buy", "symbol": "AAPL", "price": 150.0, "amount": 100, "status": "Open"}

t1.s1 end_turn: The order has been successfully placed. The details of your order are as follows: - Order ID: 12446 - Order Type: Buy - Symbol: AAPL - Price: [...]

t2.s0 ok_call: <tool_call> {"name": "cancel_order", "arguments": {"order_id": 12446}} </tool_call>

→ {"order_id": 12446, "status": "Cancelled"}

t2.s1 end_turn: The order has been successfully canceled. The details of your order are as follows: - Order ID: 12446 - Status: Cancelled Your transaction for [...]

analysis/worked_examples.py: copied from the step logs (seed 1, sampled), runs 6d0869af52b4 and 1b0b9b605f4f; task multi_turn_base_122; texts cut mechanically. Development data.

Example from our logs: a τ² retail episode, BF16 vs Q4_0

BF16: verdict fail (tau2:user_stop:reward_0)

t0.s0 reply: The process for exchanging items in a delivered order is as follows: 1. **Order ID**: #W2378156 2. **Items to Exchange**: - Mechanical keyboard ([...]

t1.s0 exec_error: <tool_call> {"name": "find_user_id_by_email", "arguments": {"email": "#W2378156"}} </tool_call>

→ Error during execution: Error: User not found

t1.s1 reply: The order ID "#W2378156" was not found in the database. Could you please check if there's a different order ID or provide more details?

t2.s0 exec_error: <tool_call> {"name": "find_user_id_by_name_zip", "arguments": {"first_name": "W2378156", "last_name": "User", "zip": "12345"}} </tool_call>

→ Error during execution: Error: User not found

t2.s1 reply: The order ID "W2378156" was not found in the database. Could you please check if there's a different order ID or provide more details? If you can conf [...]

t3.s0 exec_error: <tool_call> {"name": "find_user_id_by_email", "arguments": {"email": "#W2378156"}} </tool_call>

→ Error during execution: Error: User not found

Q4_0-pure: verdict fail (tau2:user_stop:reward_0)

t0.s0 reply: Hello! I can help with your order #W2378156. To proceed with the exchanges: 1. **Mechanical Keyboard Swap**: - You need a clicky switch keyboard. [...]

t1.s0 reply: Certainly! Let me help with that: For the mechanical keyboard: - If there is a full-size, RGB backlit model with clicky switches, that's your first c [...]

t2.s0 reply: Sure! Let me confirm that these are the correct items: - **Mechanical Keyboard**: - Replace with a full-size, clicky-switch model (RGB backlight pr [...]

t3.s0 reply: Sure! If you need to verify your account, you can check the email you provided when placing the order. If you have a device or account in mind, let me [...]

t4.s0 reply: Sure! If you can provide the order number and your shipping address, I can look up the account details for you. If that’s not possible, I can also hel [...]

t5.s0 reply: Of course! I can look up the email address for you using the order number and shipping address. If that’s not possible, I can also help confirm your i [...]

analysis/worked_examples.py: copied from the step logs (seed 1, sampled), runs fd72f54b73cb and cb673093d328; task 0; texts cut mechanically. Development data.

Eleven decisions we need before we freeze the analysis plan

Change the pre-registration

2 Step-level failures primary; episode success secondary

11 Success-based inclusion only for the secondary analysis; all 501 tasks for step-level tests

3a Add "missing call with a direct answer" to codebook v0

3b Own class for a tool that returns an error payload

8 Depth bins by step index: 0–1, 2–3, 4–6, 7+

Fix the setup

1 Context 40,960, 4,096 new tokens per step (bfcl parity); update FR-E3

4 τ^2 simulated user as a cluster job; drop the GCP L4 request

5 References: 4 sampled trials + 1 greedy pass

6 τ^2: only tasks whose gold actions write (201 of 278)

7 Backend a recorded factor (parity < 99%)

Schedule and scope

9 Keep M4 on Dec 18

10 Vision-language extension: optional 295B scope, go/no-go at M6

If you already decided any of these on Oct 30, that decision stands; tell us and we will update the record before we file the pre-registration.

Proposals by Ekant in docs/m2_decisions.md (2026-11-08), each awaiting your confirmation.

Plan to M3 (Dec 4): damage map v1

Ekant

- 4-bit damage runs as Slurm arrays (to Dec 4)
- R1 grammars per tool-call format
- Second labeler for the 300 pilot steps

Vritika

- 300 labeled pilot steps and κ → codebook v1
- Damage map v1 (Dec 4)
- OSF filing once you confirm the decisions

Yashashav

- R2 variants by Nov 22: Q5_0, Q6_K; layer-selective 5.5 bits
- IQ4_XS with an importance matrix; KL on CPU and CUDA
- QAD data (critical path); adjudicate label disagreements

M3 exit: Q4_0, Q4_K_M and IQ4_XS of every model, KL-checked on both backends; grid run as Slurm arrays; codebook v1 with κ; step-type and depth annotations.

Since Nov 9: the SmolLM3 and LFM2.5 prompt handlers are done (their chat templates); the three new models' BF16 references are complete, and the 1.7B / LFM2.5-1.2B damage cells are running or queued (counts on slide 2). Open: owners from next week (team meeting).

Workbook 2 sprint plan (Sprint 6, Nov 9–22; Sprint 7, Nov 23–Dec 6), work packages H1, H2, I, J1, K.

Dec 4: four Canvas items, and the demo we propose

Report: first two chapters

Group · locks at the deadline. Ch. 1 Project Overview, Ch. 2 Architecture. Draft to you Tue Nov 24.

Early prototype demo

Group · graded by you. Proposal: a 5–10 min recording of one run spec → Slurm array → step log → classifier → damage-map figure.

Project contribution

Individual · each of us, from the git history and meeting notes.

Meeting log 2

Group · link to the /meetings folder (Oct 1–29 notes).

Nov 24
Chapter draft to you

Dec 3
Record the demo

Dec 4
Submit all four

Dec 11
Project Expo

Canvas assignments 7852172, 7852164, 7852170, 7852168; all due Fri Dec 4, 11:59 PM. Project Expo Fri Dec 11, 10:30 AM.

What we would like from you today

1. Confirm the proposed decisions (docs/m2_decisions.md), above all 2 and 11, plus Oct 30 asks #1 and #4
2. Which failure classes are confirmatory? Proposal: any failure, exec_error, tool_error_payload
3. CPU backend: a full CPU arm does not fit the cluster. Which subset?
4. 2 GPU nodes per user: may we each run a share of the grid, or can you ask for more?
5. OSF filing: who files, under whose account, and may we list you?
6. Dec 4: live demo or recording, and when do you want the chapter draft?
7. "Use examples for all modalities": one per benchmark, domain and failure class, or a second modality?
8. Next meeting: Nov 27 is the day after Thanksgiving. Which slot instead?
9. Canvas still lists our title as "KV Cache Behavior…": how do we get it changed?
10. τ^2 simulator rules (decisions 15, 16): keep SmolLM3's 46 empty-reply episodes excluded, or score them as failures? Retry, then exclude, a simulator HTTP 500?
11. FR-E2 for SmolLM3 is 2,455 / 2,456 (decision 17): on one malformed tool call our runner fails the episode, the re-scoring drops the call. Align the re-scoring with the runner and rerun?

Appendix: cluster checks behind M2

580 / 580

S7: training steps with losses identical to the uninterrupted run after a walltime cut and requeue, across two nodes

104 / 104

S10: episodes after every array task was cut and resumed; no missing or duplicate episodes

PASS

Clean clone under Ekant's account: experiment hash 3e3a25e70f, 253 specs equal

- S7 under a 10-minute limit: B cut at step ~368, resumed from the step-365 checkpoint; max loss difference 0.0. Peak GPU memory 18.8 / 19.9 GB of 40 GB.
- S10 needed one resubmission: R1 with the forced grammar on Q4_0 used all 5 requeues with 3 episodes left; the second array ran only those 3. Size slurm.time from the slowest configuration.
- Bad nodes: A100 cs001 (GPUs 1–3 fail CUDA init) and cs002 are excluded from A100 jobs. Requeue probe: JobRequeue = 1, USR1 60 s before the limit.

results/m2/s7_compare.json, s10_check.json, s10_check_first_submission.json, clean_clone_ekant.json; docs/cluster.md.